

AI Agents and Routing

Agent anatomy · Autonomy · ReAct · Tool calling · Memory · Semantic routing

1. Topic Overview

This session covered the concept the mentors called the single most important idea in the whole cohort: the AI agent. It ran in two halves. Dhaval opened with the theory, working from a plain definition of an agent up through autonomy, guardrails and the split between agentic applications and workflows. Sid then took over for a long hands on stretch where the cohort built a working finance research agent in Google Colab, gave it memory, and finished by adding a semantic routing layer in front of it.

The session opened with a recap check on the previous weekend's material: Docling for parsing awkward PDFs and creating chunks, and LangChain as a framework for building LLM applications. Both were used again today, so the recap was not filler.

Area covered	What was taught
Agent anatomy	Agent equals LLM plus tools plus knowledge plus memory plus autonomous action. Built up from the ChatGPT flight price example and the Amazon Rufus shopping example.
Action vs tool calling	The carpenter analogy. An action is the goal handed to the agent. Tool calls are the steps the LLM makes to reach it.
Autonomy and guardrails	The cow and rope analogy. Agency and predictability move in opposite directions. Why guardrails are an engineering duty, not a nice to have.
Agentic vs workflow	Two kinds of LLM application. A news summariser is a workflow, not an agent, even though it calls an LLM.
ReAct	Reason, act, observe, repeat. The loop that sits under almost every agent framework.
Hands on: tools	Writing Python functions, annotating them with the LangChain tool decorator, and binding them to an LLM.
Hands on: agent	Building FinBot, an equity research agent, with the LangChain create agent function and a structured system prompt.
Hands on: memory	Short term vs long term memory, checkpoints, thread IDs, and moving from an in memory saver to SQLite.

Area covered	What was taught
Routing	Intent based and complexity based routing, the semantic router library, threshold optimisation, and accuracy measurement.
Quiz	An eight question Slido quiz closed the session.

Industry Context

Dhaval framed agents as the reason the whole market is moving right now, and pointed at how fast it is turning over. OpenClaw was the talked about autonomous agent a short while ago, then Hermes, and attention had already shifted again by the time of this session. His conclusion was that tool names churn but the core ideas do not, so the cohort should learn the concepts rather than chase the tools.

2. What an AI Agent Actually Is

2.1 Starting from a wrong answer

Dhaval asked the cohort for a short, precise definition of an AI agent. The most common answer in chat was "an LLM with access to tools". His response was that this is correct but incomplete, and the missing piece was the one Abhiram had typed: action.

He demonstrated the gap with ChatGPT. Ask it for flight prices from New York to Chicago for tomorrow and it answers, even though the underlying model has a training cutoff and its knowledge is frozen in time. It answers because it searches the web, that is, it calls a tool. But that system is still question and answer. It fetches information and hands it back. Sharath S put the objection in chat before Dhaval got there: it cannot take actions, it is still Q and A.

2.2 The Amazon Rufus example

The second example was Amazon's Rufus shopping assistant. You ask for organic honey rated above 4.5 stars and under \$20. It queries a products database and a reviews API, filters, and shows you a list. You then say "add it to the cart" and later "check out", and it places the order. Two things make this different from the ChatGPT case. First, it acts on the world instead of only reporting on it. Second, it holds memory of your earlier turn, so "buy the first one" resolves to a specific product.

Dhaval was careful about where memory lives. The LLM itself has no memory. It carries knowledge baked in during training, but it does not remember your previous questions. Memory is something you attach from the outside.

Key Insight

Agent equals LLM plus tools plus knowledge plus memory plus autonomous action. An agent is a program that takes input, thinks, and acts to complete a task autonomously, using tools, memory and knowledge. Dhaval asked the cohort to stop and absorb this one because everything later in the cohort builds on it.

2.3 Action is not the same thing as tool calling

Sharath S asked the sharpest question of the first hour: how is action different from tool calling? Dhaval answered with a carpenter. You call a carpenter and say "build me a chair". That instruction is the action. The carpenter then picks up a hammer, a screwdriver, a saw and nails. Each of those is a tool call. Tool calls are made by the LLM in service of the action, and the action is the higher level goal the user gave the agent. Sid added the same point more tersely in chat: a tool call happens once, while an agent can make several tool calls, with autonomy, to perform one action.

2.4 Autonomy, and the cow on a rope

Autonomy is the second half of the carpenter analogy. You give the carpenter dimensions, a material and a budget. You do not tell them how to cut the wood or how to paint the chair. They are autonomous inside your constraints. Dhaval noted that autonomy still allows clarifying questions, which is exactly what Claude Code does when it asks whether it can read your files or whether you want FastAPI or Flask.

He then drew a cow tied to a pole in a field, with food and water inside the circle the rope allows. The cow decides when to eat, drink or rest, so it is autonomous. It cannot walk onto the road and it cannot wander into the neighbour's crop, because the rope bounds it. The rope is the guardrail. Goutham named it in chat before Dhaval did.

Key Insight

Guardrails are not optional decoration. Dhaval recounted a recent industry incident, raised in chat by Anurag Pandey and several others, in which an OpenAI model being trained for cybersecurity work broke out of its guardrails and attacked the Hugging Face website. His framing: an agent with autonomy can drop a production table, move money, or post something that damages your brand. Keeping it inside the fence is the AI engineer's duty. A full session on guardrails is planned.

2.5 Agency and predictability pull against each other

The extent to which an agent can act freely is its agency, or autonomy. As agency goes up, predictability goes down. Dhaval plotted familiar tools along that line, roughly in this order.

Application	Where it sits
Traditional ChatGPT	Low agency, high predictability. A question and answer chatbot. Connect it to Gmail or Slack and it starts moving up the scale.

Application	Where it sits
n8n, Zapier and similar	Automations that perform autonomous actions on your behalf, inside a flow you defined.
Claude Code	High agency, especially if you grant bypass permissions, at which point it stops asking and keeps building.
OpenClaw, Hermes	The most autonomous end of the scale, and the fastest moving part of the market.

3. Agentic Applications vs Workflows

Dhaval split LLM applications into two families. The first is agentic. The second, which Anthropic named workflow, is everything else. He said he personally prefers the plainer term "non agentic application", and noted there is no settled industry standard here.

3.1 The news summariser that is not an agent

The worked example was the quick summary box that Indian news sites now put at the top of an article. Imagine you are the AI engineer at a news channel building it. You take the article text, run some preprocessing to strip HTML tags, author metadata and other noise, and hand it to the LLM with a prompt like "generate a three point summary of this article". You post process the result and render it.

The cohort was almost unanimous that this is not agentic, and gave the right reason: no action. The LLM has a single well defined task. It is not doing multi step planning, it is not choosing tools, and it is not acting on anything. Dhaval added the point that trips people up: even if an LLM does call a tool, that alone does not make the system agentic. A chatbot that calls tools but does not act for you is a tool augmented chatbot. The same chatbot that actually books your flight is an agent.

He also flagged that preprocessing and post processing are where input and output guardrails usually live, masking sensitive information on the way in and checking for toxicity on the way out. That is a later session.

Key Insight

Tool calling is not the test. Autonomous action is. Ask whether the system is pursuing a goal through multi step reasoning and acting on the world, or whether it is executing one well specified task you handed it.

3.2 Anthropic's guide, and the workflow patterns

Dhaval shared Anthropic's article on building effective agents in chat and walked the cohort through its diagrams. Workflows are systems where LLM calls are orchestrated through predefined code paths, each LLM having a specific task and little autonomy. Agents are the other family. Within workflows he pointed out several named patterns.

Workflow pattern	Shape
Prompt chaining	One LLM call feeds the next, with a gate in the middle that passes or fails the intermediate result.
Routing	A first LLM classifies the incoming request and sends it down a specific path. A customer support bot splitting technical support, payments and refunds, and sales enquiries into separate handlers is the classic case. This pattern returns in section 6.
Parallelisation	Several LLM calls run at once and an aggregator combines their outputs.

3.3 The ReAct loop

Agents have an intrinsic behaviour called ReAct, short for reason and act. A request comes in, the LLM thinks, it takes an action, it observes the result of that action, and it reasons again from what it observed. The loop continues until the goal is met.

Dhaval made this concrete with Claude Code, which everyone in the room uses. It writes code, runs it, notices the output does not match the requirement, says so, and rewrites the file. That self correction is the observation step feeding back into reasoning. Sid later shared the ReAct paper in chat (arXiv 2210.03629) and noted, to the cohort's surprise, that it was written by interns at Google. He placed its arrival in early 2023, which is when ChatGPT gained search, tools and memory and, in his words, became an agent.

Key Insight

The ReAct loop ends when the agent stops calling tools. There is no separate termination signal. Once the model has enough information and issues no further tool calls, it emits the final response and the loop closes. You can also cap it explicitly with a maximum iteration count, which is the answer to several chat questions about runaway token spend.

4. Hands On: Building a Finance Research Agent

Sid took over here. The cohort worked in a shared Colab notebook (agentic_ai_langchain_with_sem_router_updated.ipynb), posted in chat and also available in Discord. The goal was a finance research analyst agent that can pull stock fundamentals, search news, look things up on Wikipedia, and reason across all of it. Sid confirmed the cohort stays on Colab for now and will move to VS Code once the group has enough grounding in LLM applications and agents.

4.1 Setup

Item	Detail
Libraries	langchain, langchain-groq and langgraph. LangGraph was installed early for a small purpose that pays off later.
Model	GPT OSS via Groq, using ChatGroq. Groq was chosen because it is free and convenient. No GPU is needed: inference happens on Groq's servers.
Reasoning format	Set to parsed, because the model emits thinking tokens that need handling.
Key settings	temperature 0 and max retries 2, so a failed first generation gets one more attempt.
Common setup trap	After pasting the Groq API key into the Colab prompt you must press Enter, otherwise the cell spins forever. Sid had to repeat this twice.

4.2 Function calling and the tool decorator

Sid started from what function calling actually is. Any function call can be represented as JSON: the name of the function, plus a dictionary of argument names and values. That is all the LLM has to produce. If your model can reliably generate that JSON, it can make tool calls. The first tool was `get_stock_fundamentals`, which takes a ticker (the stock symbol, NVDA for NVIDIA, GOOGL for Google), calls the `yfinance` library, and returns price, P/E ratio, market cap, revenue growth, and the 52 week high and low as JSON. Three things turn a plain Python function into a tool.

- **Type hints** on the arguments and the return value, so the model knows the shapes involved.
- **A docstring** describing what the function does, what it takes and what it returns. Sid asked the cohort what the docstring is for now, and CHAITHANYA gave the line of the session: our colleagues are LLMs now. The docstring used to be for the developer sitting next to you. It is now read by the model.
- **The tool decorator** from LangChain, which registers the function in LangChain's tool registry.

Registration alone does nothing. You then call `bind_tools` on the LLM instance with a list of tools. Internally LangChain pulls the name, the description and the parameter and return shapes out of each tool and writes them into the system prompt, telling the model it has access to these tools and should use them when needed. Krishna clarified an important follow up in chat: the decorator only makes the function discoverable. The function executes only when the model decides to call it.

Key Insight

The LLM does the reasoning and the tool selection. The function does the deterministic execution. That division is why a legacy function you already have can become a tool with nothing more than type hints, a good docstring and a decorator.

4.3 A tool augmented LLM is still not an agent

Running the bound model against "how is NVIDIA performing right now" produced a tool call and a tool result: price, P/E, market cap, revenue growth, 52 week range. Members of the cohort pasted their own results into chat for NVDA, GOOGL, DELL and others. Some tickers failed. Amit Ashok Kulkarni's query for AIRTEL returned a 404 from Yahoo because AIRTEL is not a valid symbol, which is a useful reminder that the model guesses the ticker from its pretraining and can guess wrong.

Sid then asked whether what they had built was an agent. The cohort said no, correctly, and named the reason: agency is missing. It answered a question using a tool. It did not pursue a goal.

4.4 Four tools, one system prompt, one agent

The cohort then added three more tools, all following the same pattern of core logic, type hints, docstring and decorator.

Tool	What it does
Stock fundamentals	yfinance lookup returning price, P/E, market cap, revenue growth and the 52 week range. No API key needed, because yfinance uses Yahoo's public endpoints under a free rate limit.
Google News search	SERP API through the requests library, querying Google news for the past 24 hours and joining the results into a readable paragraph for the model.
Yahoo Finance news	yfinance news for a ticker, iterating the items and concatenating title and content into one block of context.
Wikipedia search	The wikipedia Python library, fetching the top three results and their summaries. Free, no key required.

SERP API is the only one needing a key. You create it at the SERP API dashboard and paste it into the notebook. Aditya Bharadwaj flagged that a phone number is now mandatory during Indian sign up, which Sid attributed to abuse of the free tier. His fallback: use Tavily or any other search library instead.

The system prompt is where the agent gets its shape. Sid walked through his line by line and told the cohort to pay attention, because this is what a real agent system prompt looks like. His had four parts: a role and personality (FinBot, an expert equity research analyst), the tasks it needs to accomplish, ground rules to constrain it, and an explicit output format. He mentioned the CRAFT acronym as a way to remember the parts, recalling it loosely in session as Context, Role, Action, Format and Task.

With the tools list, the LLM instance and the system prompt in hand, one call to LangChain's create agent function produces the agent. Internally, that function wires up the ReAct loop described in section 3.3. Sid noted the cohort will implement that loop from scratch in LangGraph next week.

4.5 Reading the trace

Invoking the agent with "give me a quick analyst brief on NVIDIA, is now a good time to buy" and printing the trace showed the loop clearly. One human message goes in. Then a sequence alternates: an AI message containing a tool call, a tool message containing the result, another AI message, another tool message, and so on, until a final AI message that follows the output format set in the system prompt.

Sid used the trace to check the cohort on LangChain's four message types, which they answered correctly.

Message type	Role
System message	The instructions and tool descriptions. Assembled internally by LangChain.
Human message	The user's input. In the demo there was exactly one, the goal.
AI message	The model's output. Either a tool call or the final answer.
Tool message	What a tool returned, fed back to the model as observation.

The cohort ran the agent against companies of their choice and pasted the briefs into chat: NVDA, AMD, TSLA, AAPL, GOOGL, DELL, NFLX, PFE, SHEL and INFY among others. The point of showing so many was that one goal produced multiple tool calls and multiple reasoning steps, none of them sequenced by hand.

Key Insight

You do not tell the agent which tools to call or in what order. You give it access and a goal. In one of the follow up runs the agent already had both the NVIDIA and AMD reports in context, so it compared them and called no tools at all. Deciding not to act is also agency.

5. Memory

5.1 Finding the gap

Sid deliberately broke the agent before fixing it. After the NVIDIA brief he sent a follow up: "now compare it with AMD". The agent returned a standalone AMD analysis with no reference to the earlier report. The chat diagnosed it in seconds: memory is missing, it is stateless, no previous context. Each invoke call is isolated, and the agent does not save the state of past messages unless you attach a memory layer.

5.2 Two kinds of memory

Type	What it covers
Short term	One chat session. The equivalent of a single ChatGPT window: a sequence of human, AI and tool messages that the model can see. Open a new window and the context is gone. Implemented with checkpoints.

Type	What it covers
Long term	Persists across sessions. The behaviour you see when you tell ChatGPT or Claude to remember something and it says memory updated, then recalls it in a different chat days later. Implemented with memory stores. Deferred to a later session.

Sunil offered the cleanest analogy of the session in chat: short term memory is like RAM, long term memory is like a hard disk.

5.3 Checkpointers and thread IDs

Adding short term memory took two changes. First, pass a checkpointer to the create agent call, instantiated as an in memory saver. Second, pass a config dictionary containing a thread ID when invoking the agent.

Sid tied the thread ID to something everyone had seen: the UUID at the end of a ChatGPT URL. That UUID identifies the chat session. A LangChain thread ID does the same job. If the thread ID matches, the agent looks up the past conversation for that thread, pulls those messages, and includes them in what it sends to the model. Re running the same experiment with a shared thread ID gave the right behaviour: "how does it compare to AMD on valuation" now resolved against the earlier NVIDIA report without NVIDIA being named, and a third question about growth prospects compared both.

Several people asked who generates the thread ID and how to scope it per user. Sid's answer, which he repeated in different words to Harendra and Ankit Garg, was to think like an engineer: this is an ordinary backend design decision, no different from assigning post IDs in a social app. Your application creates the session, generates the ID and passes it down. The AI layer only needs the thread ID.

5.4 Making it persistent

The in memory saver holds everything in process, so restarting the script wipes it. Swapping in the SQLite saver from langgraph checkpoint sqlite fixes that: open a SQLite connection, pass it to the saver, pass the saver as the checkpointer, and the conversation persists to a database file. The same experiment then behaved identically across restarts.

Anurag Pandey asked who defines the database schema. Sid's answer was that you do not: LangChain manages it, in the same way Keycloak or Celery create and manage their own tables once you hand them a Postgres or Redis connection. LangChain also ships integrations beyond SQLite, including Postgres, MongoDB, Redis and Oracle. Priya asked whether SQLite is used for checkpoints in real production systems, and Krishna's answer in chat was that production generally means PostgreSQL, Redis or MongoDB.

Key Insight

Sid set a homework here: replace the in memory saver with a persistent one, SQLite or Postgres, and confirm the conversation survives a restart. The only line that changes is the checkpointer instance.

6. Routing

6.1 Why routing exists

After the break, Sid opened with a question: what does the word router mean to you? The chat came back with gateways, direction, navigation and web URL routing, which was exactly the ground he wanted. His example was a travel booking assistant. A user asks "how do I change my seat in my flight from Delhi to New York". Behind the assistant sit several distinct workflows: new booking, existing booking, and technical support. Each has a genuinely different shape. New booking creates a record and asks for details from scratch. Existing booking reads a record and updates it. As a human you can tell instantly which one this question needs.

The hard part is that traditional applications never had to solve this. In a web or mobile app the user declares their intent by clicking a button, and routing is algorithmic. In an AI application the intent arrives as free text. The cohort supplied the answer: route on the semantic meaning of the query. That is intent based routing, also called semantic routing.

6.2 Two flavours, one architecture

Flavour	Example from the session
Intent based	The booking assistant. Route on what the user is trying to do: new booking, existing booking, tech support.
Complexity based	Route on how hard the request is: simple, document lookup, or complex workflow. Sid noted this is what coding agents do in their auto mode, picking a model to match the task, and confirmed it is a standard architecture in current use.

He tied it directly back to the previous session's work. The cohort had built two RAG pipelines, one answering FAQ questions from documents and one running SQL RAG. Something has to decide which pipeline a question belongs to, and that something is a router.

Key Insight

You can use an LLM to classify routes, and it works. But you pay tokens on every single request just to decide where it goes. An embedding model does the same classification with no token cost, which is why the semantic router approach exists. Satyam summed it up in chat: an LLM can also act as a router, it just costs more.

6.3 How the semantic router works

The library used was semantic router, an open source project Sid described as fast, widely used, and something he has run in production. The mechanism is simple enough to state in a sentence: you define routes, give each route example queries, and the library embeds those examples and stores them. When a

user query arrives it is embedded and matched against that store. If five neighbours come back and three of them belong to route two, the query is route two.

The pieces in code were three imports and three objects.

Piece	What it is
Encoder	A Hugging Face encoder wrapping the Qwen embedding 0.6B model. This is what turns text into vectors. No LLM is involved in routing at all.
Route	A name plus a list of example queries, called utterances. Three were defined: stock analysis, market news, and general finance.
Semantic router	Built from the encoder and the list of routes. Setting auto sync to local writes the embeddings to disk so they are not regenerated on every application start.

Sid was blunt about what determines quality: how well the router works depends entirely on the examples you provide for each route.

6.4 Measuring and improving it

Tested against a handful of queries the untuned router got most right and one wrong: "what is Apple's current P/E ratio" was classified as general finance when it should have been stock analysis. Sid used that single error to teach threshold optimisation.

Every route carries a cosine similarity threshold, defaulting to 0.5. That default is the library author's choice, not a law. Krishna explained the trade off in chat: a lower threshold accepts more queries as matches, raising the chance of routing to the wrong workflow, while a higher threshold only accepts very close matches.

Step	What happened
Training set	35 examples: 10 for each of the three routes, plus 5 out of scope questions (weather in Mumbai, write me a poem) labelled as none.
X train and Y train	Two parallel lists, questions in one and their correct routes in the other. Sid explained the naming for anyone without an ML background: X is the input, Y is the expected output.
Accuracy before	68.57 per cent, with all thresholds at the 0.5 default.
Optimisation	Calling fit on the router with the training data adjusts the per route thresholds.
Thresholds after	Stock analysis 0.74, market news 0.61, general finance around 0.65.

Step	What happened
Result	About two points of improvement on the training set, and 72 per cent on a separate held out test set with 3 wrong.

Sid was upfront that two points is not impressive and said so directly: the point was to show the mechanism, not to hit a headline number. He also defended the low number by pushing back on the cohort's assumption that this was an easy problem. Stock analysis, market news and general finance overlap heavily. Hand the same utterances to a human and they will not classify them perfectly either. The embeddings of two finance questions sit close together, so the borders are genuinely blurred. Renganathan M reached the same conclusion in chat: all of these seem closely related.

Key Insight

Asked how to reach 100 per cent accuracy, Sid said nobody knows, then described what he would actually do in production: generate a large dataset, fine tune the embedding model on it, write fresh utterances for each route, and run the semantic router on the fine tuned model. His realistic expectation from that is 80 to 90 per cent, not 100.

6.5 Wiring the router to your pipelines

The last piece is the least glamorous and the most practical. The router returns a result with a route name. You branch on it with a plain if else block: if the name is SQL, call the SQL agent; if it is FAQ, call the FAQ RAG pipeline. That is the whole integration.

Two follow ups worth recording. Queries that match nothing fall to a default route, so an off topic question does not crash the system. And when Sandip asked what happens with a hundred tools, Sid's answer was that you do not build a route per tool: you cluster tools into the agents or pipelines that use them and build one route per pipeline. Krishna phrased the same rule in chat as one router per distinct business intent or workflow, not per API or tool.

7. Mindset and How to Learn This

Two long stretches of the session were not about code at all, and both were deliberate.

7.1 Focus on the core

Partway through the tool calling section, with the chat filling up with questions about max retries, what Harness AI is, and whether yfinance covers Indian stocks, Dhaval stopped the class to give what he called the one tip behind his own ability to learn fast. When you are learning a topic, focus only on the core. Those side questions are all answerable later, by Google or in a future session. What mattered right then was how an LLM calls tools, how agentic behaviour works, and how the model processes tool output. He used two images for it: cutting vegetables for a biryani without worrying about the garnish, and Arjuna seeing only the eye of the target.

His warning was concrete. In past cohorts, people whose attention wandered during a core explanation came back later with very basic questions about that same core concept. Karthik Prabhu agreed in chat and Dhaval endorsed the phrasing: focus on the core concepts rather than the code, because frameworks like LangChain have extensive documentation you can consult once your hands are dirty.

7.2 High agency

Late in the session Dhaval connected the technical topic back to a skill from session one. Agents are exciting because they have high agency: they do a lot on their own instead of waiting for you to click buttons. If an AI engineer has one skill to develop, it is the same thing. He made it concrete with a churn example. In the previous cohort he taught LangChain community. In this one, LangChain community is deprecated, and the core concepts did not move at all. What is needed is the agency to work out the replacement yourself. If Hugging Face were replaced tomorrow, you would still be downloading a model from somewhere, still embedding text for semantic search, still passing context to an LLM. The principles hold.

Key Insight

Sid's LEGO framing, given when the cohort asked him to show how to combine agents, routing and the two RAG pipelines into one application: he is deliberately not giving the finished model. You get the bricks. Assembling them is where the critical thinking that makes an AI engineer actually develops. Help is available on Discord when you are stuck, but the experimentation has to be yours.

7.3 How Sid uses coding agents

Anurag Pandey asked a question many were thinking: coding agents now write whole applications, so where do you stop them, and what should you still do yourself? Sid described his actual workflow.

- Write the skeleton yourself: the folder structure, the modules, and the core functions that matter. For everything else, define the function signature with a pass and a comment saying what it should do.
- Point the coding agent at that skeleton and ask it to generate a PRD.
- Verify the PRD in depth. This is the step he stressed.
- Once the PRD is right, ask for the task breakdown, which comes back in phases.
- Implement phase by phase, reviewing the code after each one and feeding changes back as review comments before moving on.

His contrast: hand the whole thing over in one prompt and it will generate an enormous amount of output that you then cannot review.

7.4 The career questions

The final stretch was an open Q and A on how to learn this field: how much maths is really needed, the path into research, why the code feels hard when the concepts do not, and what to do with no coding background at all. Sid answered each one at length. Those answers are captured with attribution in the Chat Insights document for this session rather than repeated here.

8. Session Output and Next Steps

8.1 What you built

By the end of the session every participant who followed along had a working ReAct agent: FinBot, an equity research analyst with four tools, a structured system prompt, persistent memory, and a semantic routing layer in front of it.

8.2 Homework set in session

Task	Detail
JSON output	Make the agent return its analyst brief as JSON instead of formatted text. Srikanth Adilapuram and Annu both got this working during the session and were asked to share their code.
Persistent checkpointer	Replace the in memory saver with SQLite or Postgres and confirm memory survives a restart.
Add an email tool	Write a Python function that takes an email address and a message and sends it, decorate it as a tool, add it to the tools list, and adjust the system prompt to send the report to the address given in the prompt. This turns the agent's action from "research and report" into something that reaches the outside world.
Convert to a project	Turn the notebook into a modular Python project with a requirements file and a .env, and try it before asking for help. Gourav was already doing this during the session.
Improve the router	Write better utterances, re run the threshold optimisation, and see how far the accuracy moves.
Assignment	A RAG assignment lands on Discord after this session, combining embeddings, the RAG pipeline from the previous session, and routing. Watch the Discord channel for it.

8.3 Reading and links shared in session

Resource	Why
Anthropic, Building Effective Agents	anthropic.com/engineering/building-effective-agents . The source of the agent vs workflow split and the workflow patterns in section 3.
ReAct paper	arXiv 2210.03629 . The loop underneath almost every agent framework.

Resource	Why
Semantic router docs	github.com/aurelio-labs/semantic-router/tree/main/docs . Covers persistence to Postgres, Pinecone and Qdrant, and saving or loading routes from a JSON file.
Session notebook	agentic_ai_langchain_with_sem_router_updated.ipynb , in chat and on Discord.

8.4 Coming up

Sid and Dhaval flagged several topics as explicitly deferred rather than skipped: building the ReAct loop from scratch in LangGraph (next week), guardrails as a full session, long term memory, multi agent systems, fine tuning including embedding models, and semantic caching in an optimisation session.

9. Closing Quiz

The session closed with an eight question Slido quiz, run with a live leaderboard. The cohort had some fun with display names, with an Andrew NG, a Tony Stark and two Dhaval Patels on the board, one of them genuine. The questions where the answer was called out are below, and they make a fair self check on the session.

Question	Answer
As agency increases, predictability tends to	Decrease, because the system has more freedom to make decisions.
What is the correct sequence in a ReAct loop	Thought, action, observation.
What is the system prompt best used for	Setting the agent's role and boundaries.
What is the role of a checkpoint in LangGraph	It saves and restores agent state, giving turn to turn memory.
Which framework is best for multi step reasoning with branching logic	LangGraph. 91 per cent of the room got this one.
What is the process of directing a user request to the right agent, tool or workflow	Routing. 95 per cent correct.
A semantic router uses what to make routing decisions	Vector embeddings.

10. Quick Revision Summary

Term	What it means
AI agent	LLM plus tools plus knowledge plus memory plus autonomous action. A program that takes input, reasons, and acts to complete a goal on its own.
Action	The high level goal handed to the agent. Performed by the agent, not by the LLM.
Tool calling	A single step the LLM takes to reach the goal, such as an API call. One action can need many tool calls.
Agency (autonomy)	How freely the agent can act. It moves inversely to predictability.
Guardrail	The boundary you set so autonomy stays safe. The rope on the cow. A hard requirement, not a nicety.
Workflow	A non agentic LLM application. LLM calls orchestrated through predefined code paths, each with a specific task. Calling a tool does not make it agentic.
ReAct	Reason, act, observe, repeat. Ends when the agent stops calling tools.
Tool decorator	LangChain decorator that registers a Python function in the tool registry. Registration alone does not execute it.
Docstring	How the model learns what a tool does. Written for LLMs now, not just for colleagues.
bind_tools	Attaches a list of tools to an LLM by writing their names, descriptions and shapes into the system prompt. The model can only choose from what you bind.
create agent	LangChain function that takes an LLM, a tools list and a system prompt, and returns an agent running the ReAct loop internally.
Message types	System, human, AI and tool. The four message kinds you see in an agent trace.
Short term memory	One session. Implemented with checkpointers. Like RAM.
Long term memory	Across sessions. Implemented with memory stores. Like a hard disk.
Checkpointner	The object that saves and restores agent state. In memory saver for experiments, SQLite or Postgres for persistence.

Term	What it means
Thread ID	The session identifier passed in config. The same idea as the UUID at the end of a ChatGPT URL.
Semantic routing	Choosing a workflow or agent from the meaning of the user's query, using embeddings rather than an LLM. Also called intent based routing.
Utterance	An example query attached to a route. Utterances are what get embedded and stored.
Threshold	The cosine similarity a query must reach to match a route. Defaults to 0.5 and is tuned per route by fitting on labelled data.
Default route	Where a query goes when it matches nothing. Stops off topic questions from breaking the system.
CRAFT prompt	A memory aid for a good agent system prompt: context, role, action, format, task.
High agency	The human version of the same idea. The one skill Dhaval singled out for AI engineers in a market this fast.